

## The core problem it starts with

The instructor writes a function that takes an order status as a plain string:

```
func changeOrderStatus(status string) {  
    fmt.Println("Changing order status to", status)  
}  
  
func main() {  
    changeOrderStatus("received")  
    changeOrderStatus("confirmed")  
}
```

This *works*, but he points out two real-world problems with passing raw strings around:

1. **Typos / no safety.** An order has many possible statuses — *received*, *confirmed*, *prepared*, *delivered*, etc. If you scatter these string literals across the codebase, nothing stops you from misspelling "*confirmed*" as "*confirmd*". The compiler won't catch it. And if you ever need to *rename* a status (say "*confirmed*" → something else), you have to hunt down every occurrence of that string manually across the whole project.
2. **No grouping.** All these related values logically form one group ("the set of valid order statuses"), but as loose strings they aren't grouped or tied together anywhere. Good practice is to keep related values defined in one place.

Enums solve both.

## Go has no built-in enum

He's explicit about this: unlike TypeScript or other languages, **Go has no native *enum* keyword**. Instead you *construct* an enum using two things you already know — *type* and *const*.

## Building block: custom types

First he introduces custom types, since enums depend on them:

```
type MyType string
```

This creates a named type `MyType` whose underlying type is `string` (conceptually similar to a TypeScript type alias). You can define your own named types on top of existing ones — and that's exactly what an enum uses.

## The integer enum with `iota`

The main pattern taught:

```
type OrderStatus int

const (
    Received OrderStatus = iota
    Confirmed
    Prepared
    Delivered
)
```

Two pieces to notice:

- `type OrderStatus int` — a custom type backed by `int`. He chose `int` deliberately (reason becomes clear via `iota`).
- The `const (...)` block — constants grouped together (the same grouped-const syntax shown in the earlier constants lesson).

`iota` is the key concept here. It's a predeclared identifier in Go that's basically an auto-incrementing integer, **zero-indexed**, that resets at the top of each `const` block and increments by one for each line. So even though he only writes `iota` once and leaves the rest blank, the values become:

- `Received = 0`
- `Confirmed = 1`
- `Prepared = 2`
- `Delivered = 3`

The subsequent constants implicitly repeat the `= iota` expression, which is why you don't have to write the values manually. He verifies this by printing them. This is why the underlying type was `int` — `iota` produces integers.

## Using the enum in the function

The function signature changes from `string` to the enum type:

```

go
func changeOrderStatus(status OrderStatus) {
    fmt.Println("Changing order status to", status)
}

func main() {
    changeOrderStatus(Received) // prints status as 0
    changeOrderStatus(Prepared) // prints status as 2
}

```

Now you **can't** pass an arbitrary string — you must pass one of the defined constants. He runs it: `Received` prints `0`, `Prepared` prints `2`.

## The benefits he highlights

- **Type safety** — only valid `OrderStatus` constants can be passed; no stray strings.
- **Readability** — `Prepared` is self-documenting compared to a loose string.
- **Single source of truth** — in a real project this enum often lives in its own package/file. When you import the function elsewhere, hovering over it tells you it expects an `OrderStatus`, and you can jump to the type to see exactly which statuses exist. With raw strings you'd just have to *remember* the valid values.
- **Autocomplete** — in VS Code, Ctrl+Space suggests the available constants, which plain strings never gave you.

## A note on naming (capitalization)

He capitalizes the constant names (`Received` vs `received`) and mentions this isn't arbitrary — capital vs lowercase first letter controls **exported vs unexported** identifiers in Go, but he defers the full explanation to a dedicated future lesson. For now he just uses uppercase.

## The string-backed enum variant

The integer values are fine, but sometimes you want the enum to actually *be* readable strings. So he shows the alternative: back the type with `string` and assign values explicitly (no `iota`, since `iota` is integer-only):

```

type OrderStatus string

const (
    Received OrderStatus = "received"
    Confirmed OrderStatus = "confirmed"
    Prepared OrderStatus = "prepared"
    Delivered OrderStatus = "delivered"
)

```

)

Now running `changeOrderStatus(Prepared)` prints `prepared` instead of `2` — you get the proper string value rather than an integer, while keeping all the type-safety and autocomplete benefits.

He also notes that with the integer version, when the code compiles, the statuses are stored/compiled as plain integers under the hood.

## The takeaway

Enums in Go are a *pattern*, not a language feature: `type` (to define a custom named type) + `const` (to define the grouped set of values), with `iota` for auto-incrementing integers or explicit string literals when you want readable values. He closes by noting this shows up constantly in real projects — order statuses, user roles, categories — anywhere you have a fixed set of related named values.

An **enum** (short for *enumeration* / enumerated type) is just a way to define a fixed, named set of allowed values for something.

The idea: some variables can only ever be one of a small, known list of options. An order's status is one of `{received, confirmed, prepared, delivered}`. A traffic light is `{red, yellow, green}`. A user role is `{admin, editor, viewer}`. A weekday is one of seven.

Instead of representing these as raw strings or magic numbers floating around your code, an enum gives each option a **name** and bundles them together as one type. Then you say "this variable is of type `OrderStatus`," and the only legal values it can hold are the ones you defined.

Why that's useful, concretely:

- **Restricted set** — you literally can't assign a value outside the defined options. `"recieved"` (typo) won't compile if `Received` is the only valid constant.
- **Readability** — `status == Confirmed` reads better than `status == 1` or `status == "confirmed"`.
- **One source of truth** — all the valid values live in one place; rename or add one and the whole codebase follows.
- **Tooling** — autocomplete can suggest the valid options, since the type is known.

The classic mental model: a variable that's a string can be *infinitely* many things; a variable that's an enum can be *exactly one of N* things you chose ahead of time.

Under the hood each named value maps to some underlying value — usually an integer (0, 1, 2...) or sometimes a string. That backing value is mostly an implementation detail; the point of the enum is the *names* and the *constraint*.

That's exactly what the transcript built: Go has no built-in `enum` keyword, so you fake one with `type + const` (+ `iota` for the auto-incrementing integers). Languages like TypeScript, Java, C, Rust, etc. have it as a first-class feature, but the underlying concept is identical everywhere.

## The concept

An enum is a type whose values are restricted to a fixed, named set. You declare the allowed options once, give each a name, and then the compiler/type system treats that named set as a distinct type.

Go is interesting here because **it has no `enum` keyword at all**. There's no `enum OrderStatus { ... }` syntax like Java, TypeScript, or Rust. Instead, Go enums are a *convention* assembled from three primitives: a custom `type`, a `const` block, and usually `iota`.

## The basic pattern

```
type OrderStatus int

const (
    Received OrderStatus = iota // 0
    Confirmed                 // 1
    Prepared                  // 2
    Delivered                  // 3
)
```

Three things are happening:

1. `type OrderStatus int` — a new named type whose underlying type is `int`. This is what makes it a *distinct* type rather than just a bunch of loose integers. A function taking `OrderStatus` won't silently accept a raw `int`.
2. The grouped `const (...)` block — defines the members together.
3. `iota` — Go's auto-incrementing constant generator.

## `iota` in depth

`iota` is a predeclared identifier that represents the **index of the current line within a `const` block**, starting at 0 and incrementing by 1 each line. It resets to 0 at the start of every new `const` block.

The reason you only write `iota` on the first line is that Go **repeats the previous line's expression** for any `const` that omits its value. So the block above is equivalent to writing `OrderStatus = iota` on every line, and since `iota` is 0, 1, 2, 3 on those successive lines, you get 0–3.

This gives you a lot of expressive power beyond plain counting:

**Skip the zero value** (often you want 0 to mean "unset/invalid"):

```
const (
    Unknown OrderStatus = iota // 0 — sentinel for "not set"
    Received              // 1
    Confirmed             // 2
)
```

**Skip values with `_`:**

```
const (
    A = iota // 0
    _        // 1 skipped
    C        // 2
)
```

**Arithmetic on `iota`** — the expression carries forward, so you can do math:

```
const (
    KB = 1 << (10 * (iota + 1)) // 1 << 10 = 1024
    MB              // 1 << 20
    GB              // 1 << 30
)
```

**Bit-flag enums** (powers of two, so they can be OR'd together):

type Permission int

```
const (
    Read  Permission = 1 << iota // 1
    Write              // 2
)
```

```

    Execute                // 4
)

p := Read | Write         // combine
hasWrite := p&Write != 0  // check

```

## String-backed enums

If you'd rather the underlying value be a readable string (handy for JSON, logs, DB columns), back the type with `string` and assign explicitly — `iota` doesn't apply here since it's integer-only:

```

type OrderStatus string

const (
    Received OrderStatus = "received"
    Confirmed OrderStatus = "confirmed"
    Prepared OrderStatus = "prepared"
    Delivered OrderStatus = "delivered"
)

```

Trade-off: string-backed is self-describing and serialization-friendly, but slightly heavier and you lose `iota`'s auto-numbering. Integer-backed is compact and fast but the raw value (`2`) is meaningless without a lookup.

## The piece the transcript skipped: `String()` method

With an integer enum, printing a value gives you `2`, not `Prepared`. The idiomatic fix is to implement the `Stringer` interface so the type knows how to print itself:

```

func (s OrderStatus) String() string {
    switch s {
    case Received:
        return "Received"
    case Confirmed:
        return "Confirmed"
    case Prepared:
        return "Prepared"
    case Delivered:
        return "Delivered"
    default:

```

```

    return "Unknown"
}
}

```

Now `fmt.Println(Prepared)` prints `Prepared` even though it's stored as `2` — you get integer efficiency *and* readable output. There's also a code generator, `stringer` (`go install golang.org/x/tools/cmd/stringer`), that auto-generates this method for you via a `//go:generate` comment, so you don't hand-write the switch.

## Validation

Because Go enums are "just ints," nothing physically stops someone from doing `changeOrderStatus(OrderStatus(99))`. If you need guarantees, add a validator:

```

func (s OrderStatus) IsValid() bool {
    return s >= Received && s <= Delivered
}

```

## Why use the type at all

The whole payoff comes from `type OrderStatus int` being a *distinct* type:

```

func changeOrderStatus(status OrderStatus) { ... }

```

```

changeOrderStatus(Confirmed) // ✓
changeOrderStatus(2)         // ✗ won't compile — untyped int isn't OrderStatus
                             // (well, an untyped constant 2 would; a typed int var won't)

```

You get type safety, autocomplete on the constants, a single definition site, and self-documenting call sites — the four benefits the transcript demonstrated.

## Putting it together

The full idiomatic Go enum looks like this:

```

package order

```

```

type Status int

```

```
const (
    Unknown Status = iota
    Received
    Confirmed
    Prepared
    Delivered
)

func (s Status) String() string {
    return [...]string{"Unknown", "Received", "Confirmed", "Prepared", "Delivered"}[s]
}

func (s Status) IsValid() bool {
    return s > Unknown && s <= Delivered
}
```

So in short: Go has no native enum, so you build one from **type** (a distinct named type) + **const/iota** (the named members), and optionally add a **String()** method for readable output and an **IsValid()** check for safety. That convention *is* "enums in Go."